ICMP and Diagnosing a Network

Lecture 22

Computer Networks and Distributed Programming

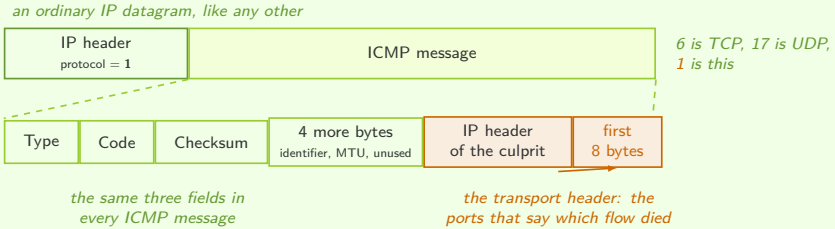
The messages we kept postponing

- We left the network layer with a promise: a router that cannot deliver a datagram drops it and tells the source.
- That telling is not a courtesy of IP. It is a protocol of this layer in its own right, and today we read it.
- Three loose ends come back with it — who says that nobody is listening on a port, what a port scanner is actually reading, and which message names the MTU that would have fitted.
- Today: the protocol, the two programs everybody already uses without knowing what they send, and a method for finding out where a network is broken.
- The method is the point. The commands are ten minutes; knowing which question to ask next is the rest of the lecture.

Talking about the network, not over it

- ICMP: the protocol with which hosts and routers exchange network-layer information about the network itself.
- Nothing of yours travels inside it. No application asks for it, and no application is the sender.
- You have read its output for years: *destination network unreachable* in a browser began as an ICMP message from some router that could not find a path.
- It exists because IP promises nothing and, left alone, reports nothing either: a datagram that dies dies silently.
- And not everything it carries is an error. Two of its messages are a question and its answer, and on those two rests `ping`.

Above IP, and carried inside it



- ICMP sits just above IP and travels inside it: the message is the payload of an ordinary datagram.
- The receiving host demultiplexes on the protocol number exactly as it would for a segment — 6, 17, and 1 for this.
- There are no port numbers, because there is nothing above to address: the message is for the machine, not for a process.

The error comes back with its cause attached

- Every message begins the same way: a type, a code within that type, a checksum, and four bytes whose use depends on the type.
- Then something unusual: a copy of the header of the datagram that caused it, plus that datagram's first 8 bytes.
- Why eight: because those are the bytes of the transport header that carry the two port numbers — or, for an echo, the identifier and the sequence number.
- A host may have hundreds of conversations open. Without those eight bytes it would know that something failed, and nothing more.

So an ICMP error is readable: it says what went wrong, and to which of your packets.

The messages, by type

Type	Code	Description
0	0	echo reply (to ping)
3	0	destination network unreachable
3	1	destination host unreachable
3	2	destination protocol unreachable
3	3	destination port unreachable
4	0	source quench (congestion control)
8	0	echo request
9	0	router advertisement
10	0	router discovery
11	0	TTL expired
12	0	IP header bad

Four of them do the whole of today: 8 and 0 are ping, 11 and 3/3 are traceroute. Wireshark labels type 11 *time-to-live exceeded*.

When no message is sent at all

- An ICMP error never causes another ICMP error: otherwise one malformed datagram could set two routers arguing for ever.
- Nothing is sent about a datagram addressed to a broadcast or multicast address — one bad packet, a thousand replies.
- For a fragmented datagram only the first fragment earns a message, since only it carries the eight bytes worth quoting.
- Routers rate-limit what they generate, so errors are the first thing dropped exactly when they are most numerous.
- And a firewall may remove the rest by policy.

So a missing message means nothing. Everything else today depends on that sentence.

Echo request, echo reply

- `ping` sends an echo request, type 8: a message whose only content is some bytes of your choosing.
- The destination sends back an echo reply, type 0, with those same bytes copied over unchanged. That is the whole test.
- And it answers by itself: the echo server is inside the operating system — not a process, nothing to install, nothing listening.
- Which is why a machine running no service at all still replies, and why a reply proves so much less than people think.
- What you learn is exactly this: a datagram reached that address, and something there was willing to answer.

Reading a ping

```
$ ping iana.org
PING iana.org (192.0.43.8) 56(84) bytes of data.
64 bytes from 192.0.43.8: icmp_seq=1  ttl=51  time=98.2 ms
64 bytes from 192.0.43.8: icmp_seq=2  ttl=51  time=98.0 ms
64 bytes from 192.0.43.8: icmp_seq=3  ttl=51  time=99.4 ms

--- iana.org ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
rtt min/avg/max/mdev = 98.0/98.5/99.4/0.60 ms
```

- `56(84)`: 56 bytes you chose, plus 8 of ICMP header and 20 of IP header. Nothing else is added — there is no transport header.
- `ttl=51` is what was left on arrival, not what was sent: started at 64, so the reply crossed some thirteen routers.
- `icmp_seq` counts the probes, and a gap in the numbering is how a loss becomes visible at all.

An application with no transport layer

`ping` is a program like any other. Which transport protocol does it use?

- Neither. It builds the ICMP message itself and hands it straight to IP; the reply comes back the same way.
- ICMP in both directions, out and in — there is no segment, no port, and no connection anywhere in a `ping`.

```
1 int fd = socket(AF_INET, SOCK_RAW,   IPPROTO_ICMP);  /* the classic way */
2 int fd = socket(AF_INET, SOCK_DGRAM, IPPROTO_ICMP);  /* unprivileged, on Linux */
```

A raw socket needs privilege, which is why `ping` used to be installed setuid; today Linux offers the second form to ordinary users.

The identifier that does the port's job

Two `ping` commands run at once, to two different hosts. Both replies arrive as ICMP, with no port anywhere. How does each one find its own program?

The echo request carries an identifier and a sequence number in those four bytes, and the reply copies both back verbatim. The identifier says which `ping`; the sequence number says which probe.

- So the key that sorts the traffic exists — it is simply inside the message instead of in a header of its own.
- It is also what lets `ping` match each reply to the request it timed, and so print one round-trip time per line.
- A protocol without ports still needs demultiplexing. Somebody always pays for it.

A silence is not an answer

You ping a host and nothing comes back. Is it down?

Nothing follows. The request may have been dropped, the reply may have been dropped, the host may be configured not to answer, and a firewall on the way may be swallowing either direction.

- Many networks filter ICMP as a matter of policy, so no reply is the normal behaviour of perfectly healthy machines.
- The converse is worse: a host that answers may have every service dead. Echo is answered by the kernel and proves nothing above it.
- Read it as one bit: a reply means the path works in both directions right now. Absence means you have learnt nothing, which is a result too.

Where the time goes

```
$ ping -c3 192.168.1.1                  the gateway, one hop away
rtt min/avg/max/mdev = 0.9/2.4/5.1/1.7 ms

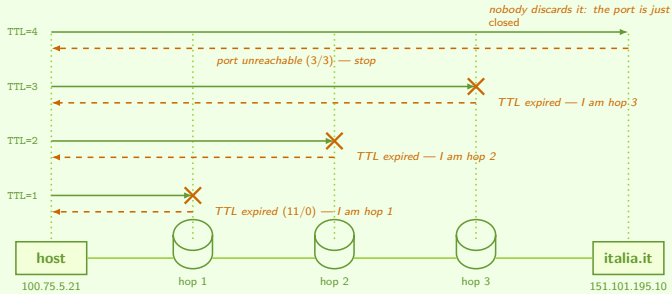
$ ping -c3 iana.org                     another continent
rtt min/avg/max/mdev = 98.0/98.5/99.4/0.60 ms
```

- The round-trip time is the sum of the four delays we already know — processing, queuing, transmission, propagation — over every hop of both directions.
- Far away, the minimum is almost pure propagation: it is a floor set by distance, and no upgrade will lower it.
- Nearby, the minimum is negligible and everything you see is queuing: same command, entirely different quantity.
- Which is why `mdev` matters more than the average. A stable 100 ms is a long path; an unstable 20 ms is a congested one.

TTL is a hop count, not a time

- The field is called time to live, and nothing in it measures time. Every router decrements it by one.
- At zero the datagram is discarded, and the router that discarded it sends back TTL expired, type 11.
- That message names its sender. So a datagram that dies is a datagram that reports where it died.
- The field was put there to stop routing loops. It turns out to be a measuring instrument, and `traceroute` is the whole idea: kill your own packets on purpose, one hop further each time.

One probe per hop



- Each probe is an ordinary datagram addressed to the final destination: only the TTL changes, and only by one.
- The source times each one, so every line of the output is an address and a round-trip time for that hop.
- Three probes per TTL, hence three times per line.

How it knows it has arrived

- Sooner or later the TTL is large enough and the probe reaches the destination, which has no reason to discard it.
- So the probes are built to be refused: each carries a UDP segment addressed to a very high port where nothing is listening (Linux starts at 33434 and counts up).
- The destination answers port unreachable, type 3 code 3 — and that message is the signal to stop sending.
- Two messages, two senders: the routers say *TTL expired*, the destination says *port unreachable*. The first draws the path, the second ends it.
- The unlikely port is not a trick played on the destination. It is the only way to make arrival distinguishable from transit.

Reading a traceroute

```
$ traceroute italia.it
traceroute to italia.it (151.101.195.10), 30 hops max, 60 byte packets
 1  _gateway (100.75.0.1)  1.785 ms  1.726 ms  2.633 ms
 2  ru-unina-l2-rl1-na01.na01.garr.net (193.206.130.9)  3.907 ms  3.869 ms  3.831 ms
 3  212.189.220.46  31.333 ms  212.189.220.218  4.657 ms  212.189.220.46  11.247 ms
 4  212.189.220.41  17.597 ms  18.875 ms  212.189.220.43  9.881 ms
 5  rs1-bo01-re1-rm02.rm02.garr.net (185.191.181.78)  27.630 ms  27.593 ms  27.556 ms
 6  * * *
```

- Hop 1 is your own gateway; hop 2 already names the network you are crossing. The names are worth more than the addresses.
- Hop 3 answers from two different addresses: the three probes did not take the same path. That is load balancing, not an error — and the same hop shows 31 ms and then 4 ms.
- Times are not monotonic along the column, because each is a fresh measurement with its own queuing.

Stars

The trace runs all the way to hop 30 without ever arriving. Did the packets get there?

Very likely yes. The destination is behind a filter that discards the probes silently, so the port unreachable that would have stopped the command never comes, and it keeps counting to its limit.

- A star is no reply within the timeout: a filter, a rate limit, or a router that simply does not generate ICMP.
- Stars in the middle, with answers after them, say only that this one hop stays quiet. The path itself is intact.
- Nor does the reverse direction have to match: what you measure is a round trip, and the way back may not be the way out.
- An address translator at a boundary is another reason a trace stops being informative — which is the next thing we look at.

Two flavours of the same idea

- UDP probes: the default on Linux, and what we have just described: the trace ends on a port unreachable.
- Echo probes: `traceroute -I`, and what `tracert` does on Windows: echo requests with a growing TTL, and the trace ends on an echo reply.
- The mechanism is identical. Only the probe and the stop condition change, and the routers in the middle answer the same way.
- It matters because the two are filtered differently: a path that swallows UDP to strange ports may answer echo, and a path that blocks all ICMP answers neither.
- When a trace looks wrong, running the other flavour is the cheapest second opinion there is.

Which process gets the reply?

The router's message is addressed to your host and carries no port. So which of your programs receives it?

None of them, by the ordinary route. `traceroute` reads ICMP from a raw socket, or asks the kernel to report errors on the socket it probed with, and recognises its own probe by the eight quoted bytes — which are the ports it chose.

- That is where the quoting rule earns its keep: without it, a reply could not be matched to a probe at all.
- So `traceroute` sends UDP and still uses no transport service: the segment is a container picked so that nobody listens.
- Two programs, the same shape. Both live at the network layer and borrow the socket API to get there.

The name resolved before the first probe

```
Domain Name System (query)
  Flags: 0x0100 Standard query
    .... ...1 .... .... = Recursion desired: Do query recursively
  Queries
    italia.it: type AAAA, class IN
```

- The first packet of the capture is not a probe: it is a DNS query, because the command was given a name.
- *Recursion desired* is set — your host asks the resolver to do the whole job and hand back one answer.
- And it happens again in reverse, once per hop, to turn each address back into a name. That is why `traceroute -n` is faster.
- So: a broken trace may be a broken name service, and the capture tells the two apart in its first two packets.

Three thousand bytes

```
$ traceroute italia.it 3000      total packet length, not payload (default 60)
```

	Data	In the field	Wireshark prints	MF
fragment 1	1 480	0	0	1
fragment 2	1 480	185	1 480	1
fragment 3	20	370	2 960	0

- The header counts the offset in units of 8 bytes; Wireshark multiplies by 8 before printing it. Its own bit display settles it: `0 0000 1011 1001` is 185, labelled 1 480.
- All three carry the same identifier and the same TTL: fragmentation happens beneath the probe and does not disturb it.

Destination unreachable, and who said so

Code	Means	Sent by
0	no route to that network	a router on the way
1	the network is there, the host is not	the last router
2	nothing handles that protocol number	the destination
3	nothing is bound to that port	the destination
4	too large, and you forbade fragmenting	a router on the way
13	administratively prohibited	a firewall

- The sender column is the useful one: codes 0, 1 and 4 come from the middle of the path, 2 and 3 prove you arrived.
- Code 3 is the one your own programs will meet, and the one that ends a trace.

Fragmentation needed, and the black hole

The datagram is too large for the next link and carries *don't fragment*. The router drops it and reports — and the report is filtered on the way back. What does the sender see?

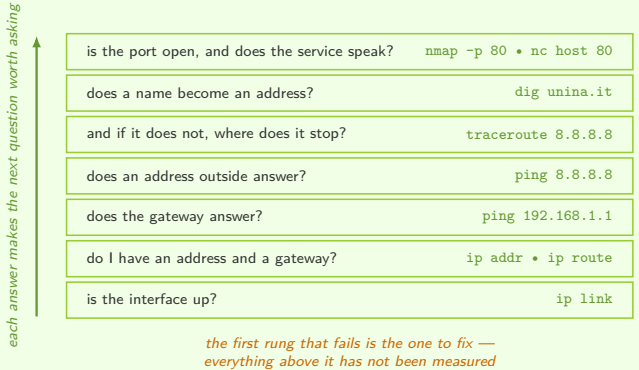
Nothing at all. It retransmits the same over-sized datagram for ever. The handshake succeeded, small messages pass, large ones vanish, and the connection hangs in the middle of a transfer.

- Type 3 code 4 carries, in its four spare bytes, the MTU that would have fitted. Path MTU discovery is built on that number.
- Blocking all ICMP therefore breaks a mechanism that TCP depends on, and the symptom points anywhere but at the firewall.
- It hides very little — a host is found by other means — and costs you exactly the messages you need when something breaks.

The ones that fell out of use

- Source quench: congestion control by ICMP: a router asking a host to slow down. It is no longer used — that job went to the transport layer, and to the two bits in the IP header.
- Redirect: a router telling a host it picked the wrong first hop. Usually ignored now, because a forged one reroutes your traffic.
- Router discovery: a host asking who the routers are. It survives, but the address itself normally arrives another way.
- The newer version of the protocol reorganises all of this and adds packet too big, which is not optional there: routers no longer fragment, so the message is the only way to find out.

A ladder of questions



- Start at the bottom even when you are sure: the two cheapest questions are the ones that fail most often.
- What ICMP contributes is that the middle rungs are answerable at all — without it, everything between you and the service is one undivided silence.

The toolbox

Command	Question it answers	Where it looks
`ip link`, `ip addr`	am I connected, and who am I	this machine
`ip route`	where does my traffic go first	this machine
`ping`	does a datagram get there and back	the path
`traceroute`	how far does it get before it stops	the path
`dig`	does the name become an address	the name service
`ss -tn`	are the sockets I expect open	this machine
`nmap`	who is up, which ports answer	the other end
`nc`	does the service actually speak	the other end
`tcpdump`	what really left this machine	every layer

None of them answers a question the one above it has left open — which is why the order is the tool, and the commands are just commands.

Who is on this network

```
$ sudo nmap -sn 192.168.1.0/24
Nmap scan report for _gateway (192.168.1.1)
Host is up (0.0024s latency).
MAC Address:  10:BD:18:E5:74:80 (Cisco Systems)
Nmap scan report for 192.168.1.17
Host is up (0.0031s latency).
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.35 seconds
```

- `-sn` asks who exists and stops there: no ports are probed.
- On your own subnet the MAC addresses give the method away — it is not pinging anybody, it is asking the link layer who owns each address. Nothing else could return a MAC.
- Off the subnet that option is gone, and the sweep falls back on echo requests plus a probe to a common port or two.
- So the same command answers a different question from inside and from outside a network, and it is worth knowing which you asked.

Open, closed, filtered — and who says so

Result	What comes back	Who sent it
open	SYNACK	the service itself
closed	RST	the remote TCP: the host is up
filtered	nothing, or ICMP 3/13	something on the way

- The third row is where this layer appears: a firewall may be polite and say administratively prohibited, or say nothing at all.
- And on UDP there is no RST: a closed port answers port unreachable, a filtered one answers nothing — so the scanner has to wait out a timeout to tell them apart.
- Which also means: the rate limit of frame 6 makes a UDP scan slow on purpose. The protocol is defending itself.

What a capture cannot see

```
$ dig +norecurse @<a root server> iana.org A      asking the root directly
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 6, ADDITIONAL: 13
org.        172800  IN  NS  a0.org.afilias-nst.info.   expected:  a referral, no address

$ dig +norecurse @<a root server> iana.org A      same command, now behind a VPN
iana.org.       340  IN  A   192.0.43.8        an answer it could not have given
```

- A root server has no address for that name and should hand back the servers of the zone. Behind one VPN provider, it handed back the address.
- Wireshark showed nothing: the traffic to the VPN is encrypted, and the capture had no packets to display.
- The explanation is that the VPN resolved the name itself and returned the answer as if it came from the server that was asked. With another provider the referral came back as expected.
- The tool did not lie: it reported what arrived. What it cannot report is who really sent it — but the header can: a root server sets `aa` and never `ra`. No VPN needed to see this: many home connections intercept port 53 all the time.

Summary

- ICMP is how hosts and routers talk about the network. It sits above IP, travels inside IP, and has no ports.
- A message carries a type, a code, and the header plus first eight bytes of the datagram that caused it — so an error names its own cause.
- `ping` is types 8 and 0, and answers one question: a datagram reached that address and something replied.
- `traceroute` kills its own probes on purpose — TTL expired from each router, port unreachable from the destination, which is how it knows to stop.
- A silence is not an answer: errors are rate-limited, filtered, and never generated at all in several cases.
- Diagnosis is an order of questions. Next: how a host gets its address in the first place, and what happens when a whole network hides behind one.

Your own client, from one layer down

```
$ ./chatd 5000 &
$ sudo tcpdump -i lo -n -X 'port 5000'     or:  filter ip.addr==...  in Wireshark
```

- Everything your chat sends is a segment inside a datagram: protocol 6 for the connected version, 17 for the datagram one.
- On loopback the TTL leaves at 64 and arrives at 64 — nothing routed it. Across the lab it arrives at 63, and the missing one is the router.
- In the payload you can read your own framing: four bytes of length, one of type, then the body. It is the first header in this course that you wrote yourselves.
- Nothing below knows what those five bytes mean. The format is an agreement between your two programs and nobody else.

The message your chat already receives

Start your datagram client with no server running. Nothing is bound to port 5000. What happens on the wire?

The host you addressed replies port unreachable, type 3 code 3 — the very message that ends a traceroute. Whether your program ever hears about it depends on one call.

```c
connect(fd, (struct sockaddr *) &srv, sizeof srv);   /* records the peer */
sendto(fd, msg, len, 0, NULL, 0);                    /* returns fine: it left */

ssize_t n = recv(fd, buf, sizeof buf, 0);
if (n < 0 && errno == ECONNREFUSED)  /* the ICMP came back, and was kept */
```

Without `connect` the kernel cannot say which socket the message concerns, and drops it: your client times out and retransmits for ever.

What `connect` tells you, and who told it

Your `errno`	What arrived	What follows
`ECONNREFUSED` (TCP)	a RST	host up, nothing on that port
`ECONNREFUSED` (UDP)	ICMP 3/3	host up, nothing bound there
`EHOSTUNREACH`	ICMP 3/1	a router could not deliver it
`ENETUNREACH`	ICMP 3/0	no route to that network
`EACCES`	ICMP 3/13	a firewall refused it, and said so
`ETIMEDOUT`	nothing at all	you have learnt nothing

- Four of the six are an ICMP message translated into an integer for you — the network layer reported through a socket call. The first is a TCP segment, and only the last is nothing at all.
- That last one is what gets reported as *the server is down*, and it is precisely the one that says no such thing.

Capture a traceroute

Start the capture, then run the command and let it finish.

```
$ traceroute italia.it
ip.src==<your address> && ip.dst==<the server> && udp     display filter
```

Select the first probe and open its Internet Protocol section.

(a) What is your host's address, and what is in the TTL field of this first probe?

(b) Which upper-layer protocol does the header declare, and by which number?

(c) How many bytes of IP header are there, and how many of payload? Say how you worked the second one out — it is not a field.

(d) Has this datagram been fragmented? Name the fields you looked at, and what they would have said if it had.

What stays the same, what changes

Look now at the whole sequence of probes, and then at what came back.

```
ip.dst==<your address> && icmp                the replies from the routers
```

(e) Across the probes, which header fields stay constant and which change? For each constant one, why must it be constant?

(f) Describe the pattern in the `Identification` field of the probes.

(g) In the datagrams the routers sent, what upper-layer protocol is declared?

(h) Are their `Identification` values related to those of your probes? Should they be?

(i) Are the TTLs of the returning messages similar to one another? Where did each of them start?

Three thousand bytes, on your own machine

```
$ traceroute italia.it 3000        total length of each probe, in bytes
ip.src==<your address> && ip.dst==<the server>
```

(j) Find the first fragment of the first probe. How many datagrams did the 3 000 bytes become?

(k) Which fields in the header say that this is a fragment at all?

(l) How do you tell a first fragment from the ones after it, and the last from the ones before it?

(m) How many bytes does each of them carry, header included? Check that they add up.

(n) Compare the bits of the offset field with the number Wireshark prints beside them. Which of the two is in the datagram?

And the ping

```
$ ping -c5 iana.org
icmp                                        the whole display filter you need
```

(o) Identify the echo requests your machine sent and the replies that match them. Which addresses are source and destination in each direction?

(p) In one echo request: what is the TTL in the IP header, what is the type in the ICMP header, and what are the identifier and the sequence number?

(q) Measure the time between a request and its reply, and compare it with the figure `ping` printed for that sequence number.

(r) Now run it against a host that does not answer. What does the capture contain, and what does that allow you to conclude?